

# RAPPORT DE PROJET

# Supply Chain Security

# Kubernetes

Scan d'images Docker et contrôle d'admission  
avec Trivy, Safety et OPA Gatekeeper

---

**Mouaad Sekkouri**

M1 Cybersécurité — Supinfo Lille  
Juin 2026

## Table des matières

1. Contexte et objectifs
2. Architecture du projet
3. Application Flask et Dockerfile sécurisé
4. Scan de vulnérabilités (Trivy et Safety)
5. Pipeline CI/CD GitLab
6. OPA Gatekeeper — Contrôle d'admission Kubernetes
7. Déploiement sur le cluster
8. Conclusion

## 1. Contexte et objectifs

Ce projet s'inscrit dans le cadre de ma formation en M1 Cybersécurité à Supinfo Lille. L'objectif est de sécuriser la chaîne d'approvisionnement logicielle (supply chain) d'un cluster Kubernetes, en mettant en place des mécanismes automatiques de détection de vulnérabilités et de contrôle d'admission.

Les attaques supply chain sont devenues l'un des vecteurs les plus critiques en cybersécurité. Des incidents comme SolarWinds (2020), Log4Shell (2021) ou la compromission de CodeCov ont montré qu'une dépendance vulnérable ou une image Docker corrompue peut compromettre toute une infrastructure.

Le principe de défense en profondeur guide l'architecture : trois barrières de sécurité successives, chacune couvrant un angle différent, pour garantir qu'une image vulnérable ne puisse jamais atteindre la production.

### Stack technique

| Composant         | Outil                | Rôle                                   |
|-------------------|----------------------|----------------------------------------|
| Application       | Flask (Python)       | Application cible à sécuriser          |
| Conteneurisation  | Docker (multi-stage) | Build d'images sécurisées              |
| CI/CD             | GitLab CI/CD         | Automatisation du pipeline             |
| Scan image        | Trivy                | Détection CVE dans les images Docker   |
| Scan dépendances  | Safety               | Détection CVE dans les packages Python |
| Orchestration     | Kubernetes (Kind)    | Cluster local pour le déploiement      |
| Admission control | OPA Gatekeeper       | Policies de sécurité au niveau cluster |

## 2. Architecture du projet

Le pipeline de sécurité se décompose en trois barrières successives qui forment une défense en profondeur :

### Barrière 1 — Trivy (scan de l'image Docker)

Trivy analyse l'image Docker couche par couche et compare chaque paquet installé (OS et applicatif) contre les bases de vulnérabilités connues (NVD, Debian Security Tracker, Alpine SecDB). Si une CVE de sévérité CRITICAL ou HIGH est détectée, le pipeline s'arrête avec un exit code 1. L'image vulnérable n'est jamais pushée vers le registre.

### Barrière 2 — Safety (scan des dépendances Python)

Safety est complémentaire à Trivy. Tandis que Trivy voit l'image comme un ensemble de paquets génériques, Safety se concentre sur l'écosystème Python spécifiquement, avec la base PyUp.io qui est souvent plus réactive sur les CVE Python. C'est le principe ceinture et bretelles.

### Barrière 3 — OPA Gatekeeper (admission controller)

Même si quelqu'un contourne le pipeline CI/CD et pousse une image manuellement via kubectl, Gatekeeper intercepte la requête au niveau de l'API server Kubernetes via le mécanisme des admission webhooks. Trois politiques sont appliquées : interdiction du tag latest, interdiction de l'exécution en root, et restriction aux registres approuvés uniquement.

### Structure du projet

| Dossier / Fichier       | Contenu                                         |
|-------------------------|-------------------------------------------------|
| app/                    | Application Flask, Dockerfile, requirements.txt |
| gatekeeper/templates/   | ConstraintTemplates (logique Rego)              |
| gatekeeper/constraints/ | Constraints (paramètres concrets)               |
| k8s/                    | Manifests Kubernetes (Deployment, Service)      |
| .gitlab-ci.yml          | Pipeline CI/CD complet                          |
| .trivyignore            | CVE acceptées (exceptions justifiées)           |

## 3. Application Flask et Dockerfile sécurisé

### 3.1 Application cible

L'application est volontairement minimaliste : une API Flask avec deux endpoints (/ et /health). L'objectif n'est pas l'application elle-même, mais la sécurisation de tout ce qui l'entoure. Le endpoint /health est utilisé par Kubernetes pour les livenessProbe et readinessProbe.

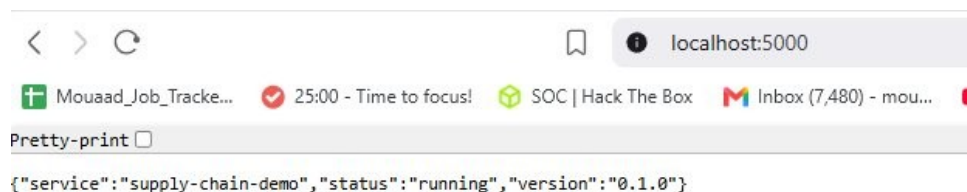


Figure 1 — Endpoint / de l'application Flask

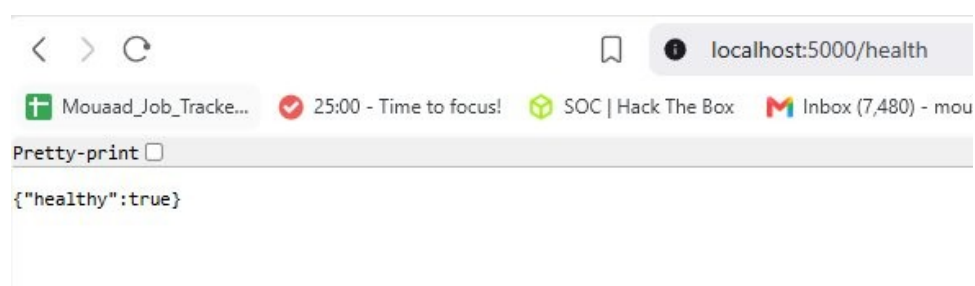


Figure 2 — Endpoint /health pour les health checks Kubernetes

### 3.2 Dockerfile sécurisé

Le Dockerfile applique trois mesures de sécurité clés :

**Multi-stage build** : l'image finale ne contient que le runtime Python et le code applicatif. Les outils de compilation, le cache pip et les headers ne sont pas inclus. Résultat : une image de 46 Mo au lieu de plusieurs centaines, et une surface d'attaque réduite.

**Utilisateur non-root** : un utilisateur appuser est créé et l'instruction USER bascule l'exécution sur cet utilisateur. La commande docker exec whoami confirme que le conteneur tourne en tant que appuser et non root.

**Serveur de production** : gunicorn remplace le serveur de développement Flask, qui n'est pas conçu pour la production.

## 4. Scan de vulnérabilités

### 4.1 Trivy — Scan de l'image Docker

Trivy a détecté 106 vulnérabilités au total dans l'image, dont 2 CRITICAL et 7-8 HIGH provenant principalement des paquets OS Debian (perl, ncurses). Le scan filtré avec `--severity CRITICAL,HIGH --exit-code 1` retourne un exit code 1, ce qui signifie que le pipeline serait automatiquement bloqué.

Le rapport JSON généré en parallèle est archivé comme artefact du pipeline pour l'audit et la traçabilité.

### 4.2 Safety — Scan des dépendances Python

Safety a détecté 6 vulnérabilités dans 4 packages Python :

| Package       | Version | CVE            | Impact                                   |
|---------------|---------|----------------|------------------------------------------|
| requests      | 2.31.0  | CVE-2024-35195 | Bypass de la vérification de certificats |
| requests      | 2.31.0  | CVE-2024-47081 | Fuite de credentials .netrc              |
| requests      | 2.31.0  | CVE-2026-25645 | Réutilisation de fichiers temporaires    |
| gunicorn      | 22.0.0  | PVE-2024-72809 | HTTP request smuggling                   |
| flask         | 3.0.3   | CVE-2026-27205 | Disclosure d'information via cache       |
| python-dotenv | 1.0.1   | CVE-2026-28684 | Écrasement arbitraire de fichiers        |

La complémentarité entre Trivy et Safety est démontrée : Trivy a identifié les CVE au niveau des paquets OS (perl, ncurses) tandis que Safety a trouvé des vulnérabilités Python supplémentaires (gunicorn HTTP smuggling, Flask information disclosure) que Trivy ne remontait pas en CRITICAL/HIGH.

## 5. Pipeline CI/CD GitLab

### 5.1 Structure du pipeline

Le pipeline GitLab CI/CD est structuré en quatre stages séquentiels :

| Stage | Job              | Action                         | Bloquant |
|-------|------------------|--------------------------------|----------|
| build | build-image      | Build Docker + sauvegarde .tar | Oui      |
| scan  | trivy-image-scan | Scan CVE CRITICAL/HIGH         | Oui      |
| scan  | safety-scan      | Scan dépendances Python        | Oui      |
| push  | push-image       | Push vers le registre GitLab   | Oui      |

### 5.2 Mécanisme de blocage

Le point critique du pipeline est le mécanisme de gate entre les stages scan et push. Les jobs Trivy et Safety utilisent `allow_failure: false`, ce qui signifie que si l'un des deux détecte une vulnérabilité, le pipeline s'arrête et l'image n'est jamais pushée vers le registre.

La capture ci-dessous montre le pipeline en état Failed : le build a réussi, les deux scans ont détecté des vulnérabilités et ont échoué, et le stage push n'a jamais été exécuté.

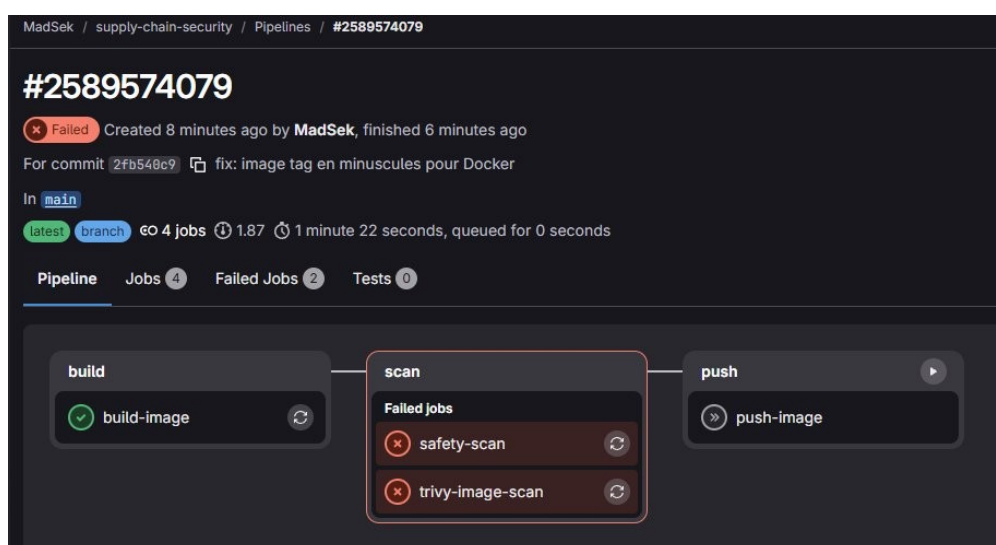


Figure 3 — Pipeline GitLab : build ✓, scan ✗ (Trivy + Safety), push non exécuté

### 5.3 Tagging sécurisé des images

Chaque image est taggée avec le SHA du commit (`$CI_COMMIT_SHORT_SHA`) plutôt que le tag latest. Cela garantit la traçabilité : on sait exactement quel code tourne en production, et un rollback vers une version précédente est trivial.

## 6. OPA Gatekeeper — Contrôle d'admission

### 6.1 Principe de fonctionnement

OPA Gatekeeper est un admission controller Kubernetes. Il s'installe comme un webhook de validation sur l'API server. Chaque requête de création ou de modification de ressources (Pods, Deployments) est interceptée et évaluée contre les politiques définies en Rego, le langage de politique d'Open Policy Agent.

Gatekeeper utilise un modèle en deux couches :

**ConstraintTemplate** : définit la logique de vérification en Rego (le comment vérifier).

**Constraint** : instancie le template avec des paramètres concrets (le quoi vérifier). C'est le même principe qu'une classe et une instance en programmation orientée objet.

### 6.2 Les trois politiques

| Policy               | Objectif                                                 | Risque couvert                                |
|----------------------|----------------------------------------------------------|-----------------------------------------------|
| K8sAllowedRegistries | Autoriser uniquement les images du registre GitLab privé | Image backdoorée depuis un registre public    |
| K8sNoLatestImage     | Interdire le tag latest et les images sans tag           | Non-reproductibilité, remplacement silencieux |
| K8sNoRunAsRoot       | Interdire l'exécution en tant que root (UID 0)           | Évasion de conteneur, escalade de privilèges  |

### 6.3 Installation et vérification

Les ConstraintTemplates et Constraints sont déployés sur le cluster Kind. La capture ci-dessous montre les trois templates et constraints installés avec l'enforcementAction deny :

```
hadsek@Travis:~/supply-chain-security$ kubectl get constrainttemplates
NAME                                AGE
k8sallowedregistries                77s
k8snolatestimage                    77s
k8snorunasroot                      77s
hadsek@Travis:~/supply-chain-security$ kubectl get constraints
NAME                                ENFORCEMENT-ACTION  TOTAL-VIOLATIONS
k8sallowedregistries.constraints.gatekeeper.sh/allowed-registries  deny                  1
NAME                                ENFORCEMENT-ACTION  TOTAL-VIOLATIONS
k8snolatestimage.constraints.gatekeeper.sh/no-latest-tag           deny                  0
NAME                                ENFORCEMENT-ACTION  TOTAL-VIOLATIONS
k8snorunasroot.constraints.gatekeeper.sh/no-run-as-root            deny                  1
hadsek@Travis:~/supply-chain-security$
```

Figure 4 — ConstraintTemplates et Constraints installés sur le cluster

### 6.4 Tests de validation

Les politiques sont testées avec des requêtes dry-run (--dry-run=server) qui envoient la requête à l'API server (donc Gatekeeper l'évalue) sans créer réellement les ressources.



```

madsek@Travis:~/supply-chain-security$ kubectl run test-latest --image=nginx:latest --dry-run=server
Error from server (Forbidden): admission webhook "validation.gatekeeper.sh" denied the request: [no-run-as-root] BLOQUE : conteneur 'test-latest' ne garantit pas execution non-root.
[allowed-registries] BLOQUE : image 'nginx:latest' ne vient pas d'un registre approuvé. Autorises : ["registry.gitlab.com/mouaadsek/"]
[no-latest-tag] BLOQUE : image 'nginx:latest' utilise le tag latest.
madsek@Travis:~/supply-chain-security$ kubectl run test-notag --image=nginx --dry-run=server
Error from server (Forbidden): admission webhook "validation.gatekeeper.sh" denied the request: [no-run-as-root] BLOQUE : conteneur 'test-notag' ne garantit pas execution non-root.
[no-latest-tag] BLOQUE : image 'nginx' n'a pas de tag.
[allowed-registries] BLOQUE : image 'nginx' ne vient pas d'un registre approuvé. Autorises : ["registry.gitlab.com/mouaadsek/"]
madsek@Travis:~/supply-chain-security$

```

Figure 5 — Gatekeeper bloque nginx:latest (3 politiques déclenchées simultanément)

Les trois politiques se déclenchent simultanément sur une image non conforme : no-run-as-root, allowed-registries et no-latest-tag bloquent chacune la requête avec un message explicite.

En revanche, un pod conforme (registre approuvé, tag fixe, runAsNonRoot: true) est accepté :

```

madsek@Travis:~/supply-chain-security$ cat <<EOF | kubectl apply --dry-run=server -f -
apiVersion: v1
kind: Pod
metadata:
  name: test-ok
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
  containers:
    - name: app
      image: registry.gitlab.com/mouaadsek/supply-chain-security:v0.1.0
      securityContext:
        allowPrivilegeEscalation: false
EOF
pod/test-ok created (server dry run)
madsek@Travis:~/supply-chain-security$

```

Figure 6 — Pod conforme accepté par Gatekeeper

## 7. Déploiement sur le cluster

### 7.1 Manifest Kubernetes sécurisé

Le manifest de déploiement applique plusieurs mesures de sécurité au niveau du pod et du conteneur :

| Mesure                  | Paramètre                           | Impact                                           |
|-------------------------|-------------------------------------|--------------------------------------------------|
| Non-root                | runAsNonRoot: true, runAsUser: 1000 | Le pod ne peut pas tourner en root               |
| Read-only filesystem    | readOnlyRootFilesystem: true        | Un attaquant ne peut pas écrire sur le disque    |
| Drop capabilities       | capabilities: drop: [ALL]           | Toutes les capabilities Linux sont supprimées    |
| No privilege escalation | allowPrivilegeEscalation: false     | Bloque les exploits setuid/setgid                |
| Resource limits         | memory: 128Mi, cpu: 200m            | Prévient le DoS par consommation de ressources   |
| Health checks           | livenessProbe, readinessProbe       | Redémarrage automatique si le pod est défaillant |

### 7.2 Preuve du fonctionnement

La première tentative de déploiement a été bloquée par Gatekeeper car l'image supply-chain-demo:v0.1.0 ne correspondait pas au préfixe du registre approuvé. Après avoir retagé l'image avec le bon préfixe (registry.gitlab.com/mouaadsek/), le déploiement a réussi avec 2 pods Running :

```

Conditions:
  Type             Status  Reason
  ----             -
  ReplicaFailure   True    FailedCreate
Events:
  Type             Reason      Age   From              Message
  ----             -
  Warning          FailedCreate 17s (x15 over 106s) replicaset-controller Error creating: admission webhook "validation.gatekeeper.sh" denied the request: [allowed-registries] BLOQUE : image 'supply-chain-demo:v0.1.0' ne vient pas d'un registre approuvé. Autorises : ["registry.gitlab.com/mouaadsek/"]
madsek@Travis:~/supply-chain-security$ kubectl delete deployment supply-chain-demo
deployment.apps "supply-chain-demo" deleted from default namespace
madsek@Travis:~/supply-chain-security$ docker tag supply-chain-demo:v0.1.0 registry.gitlab.com/mouaadsek/supply-chain-security:v0.1.0
madsek@Travis:~/supply-chain-security$ kind load docker-image registry.gitlab.com/mouaadsek/supply-chain-security:v0.1.0 --name supply-chain-lab
Image: "registry.gitlab.com/mouaadsek/supply-chain-security:v0.1.0" with ID "sha256:94e6672d7a0b803708f7d0ad309d4b1a1ff3354b34d950ecc0793c21bdb081c4" not yet present on node "supply-chain-lab-control-plane", loading.
..
madsek@Travis:~/supply-chain-security$ sed -i 's|supply-chain-demo:v0.1.0|registry.gitlab.com/mouaadsek/supply-chain-security:v0.1.0|' k8s/deployment.yaml
madsek@Travis:~/supply-chain-security$ kubectl apply -f k8s/deployment.yaml
deployment.apps/supply-chain-demo created
madsek@Travis:~/supply-chain-security$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
supply-chain-demo-78f6846c6f-wpf7p  0/1     Running   0           3s
supply-chain-demo-78f6846c6f-zcssl  0/1     Running   0           3s
supply-chain-demo-78f6846c6f-zcssl  1/1     Running   0          10s
supply-chain-demo-78f6846c6f-wpf7p  1/1     Running   0          10s

```

Figure 7 — Blocage par Gatekeeper puis déploiement réussi après correction

Cette séquence (blocage → correction → succès) démontre que Gatekeeper fonctionne comme dernière ligne de défense, même pour les déploiements effectués directement sur le cluster.

## 8. Conclusion

Ce projet démontre la mise en place d'une sécurité supply chain complète pour un environnement Kubernetes, basée sur le principe de défense en profondeur avec trois barrières de sécurité :

**Trivy** scanne les images Docker et bloque le pipeline CI/CD si des vulnérabilités CRITICAL ou HIGH sont détectées dans les paquets OS ou applicatifs.

**Safety** complète Trivy en se concentrant sur les dépendances Python spécifiquement, avec une base de vulnérabilités dédiée.

**OPA Gatekeeper** constitue la dernière ligne de défense au niveau du cluster Kubernetes, refusant tout déploiement d'images non conformes, même si le pipeline CI/CD est contourné.

Le résultat est un pipeline où une image vulnérable ne peut jamais atteindre la production : elle est bloquée au niveau CI/CD par les scans, et même en cas de contournement, au niveau du cluster par l'admission controller.

## Liens du projet

**GitLab** : <https://gitlab.com/MouaadSek/supply-chain-security>

**GitHub** : <https://github.com/MouaadSek/supply-chain-security>

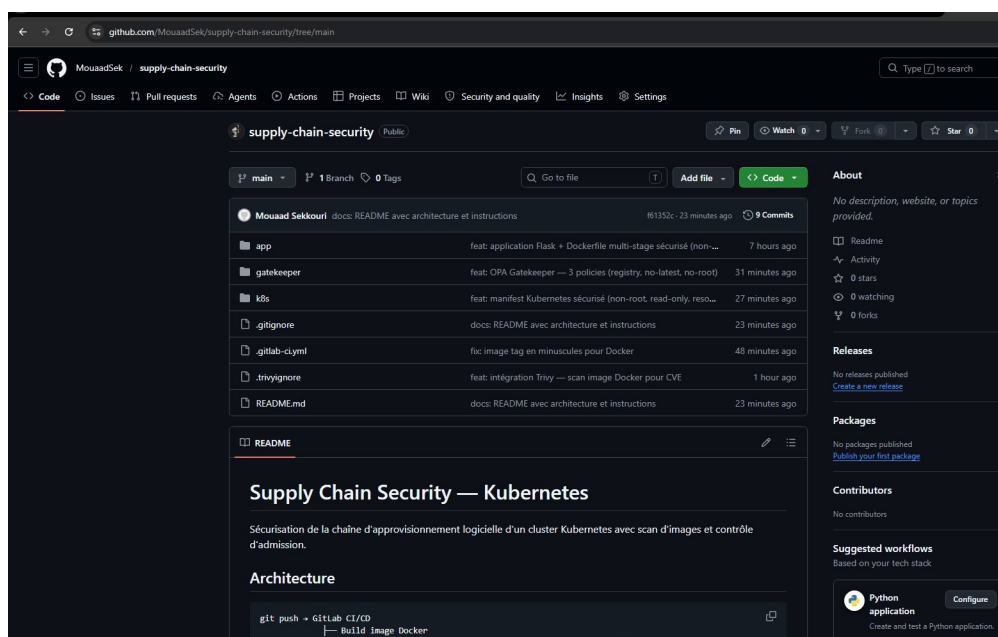


Figure 8 — Repository GitHub avec le README et les 9 commits